Revelio · 10/30 9:00 · Patrick 作业讲解 · 演讲稿

每一节对应一页 Slides。黑字是你要说的话（英文，bullet），棕字是中文提示和被追问时的补充。带 高亮 的是重点句。

Slide 0 · 开场

- Thanks for having me. I put the main diagrams in a short deck so it's easier to follow. I'll walk through it on screen and drop the link in the chat so you can keep it.
- Plan for the hour: the overall design first, then what the data told me, then the matching rules, then scale, daily updates, and how I evaluated it. Please stop me anytime.

先说流程，让他知道你有主线。他打断就停在那一页答，别跳去翻代码。

Slide 1 · The problem

- The input is 11.5 million rows of job–profile scrapes for about 150 thousand people. Each person is scraped many times, and each scrape is a partial, sometimes edited copy of the profile.
- The output is, for every person and every scrape date, the most complete work history we know on that date, with one stable id per position.
- The constraint is no future information, including the ids: what I output for January can only use what was scraped up to January.
- Two cases the brief named: Case A, a scrape misses a job that was there before, so I carry it forward. Case B, the person edits a job, say the title changes, and it has to keep the same id.

题目一句话：每人每天最完整履历 + 稳定 id + 不用未来信息。

Slide 1 补充 · 按 Patrick 题目的要求切入（他很可能希望你这么讲）

题目原文就是 7 条要求 + 评分「准确度 / 速度 / 两者平衡」。开场把这 7 条过一遍、每条一句话说怎么满足，他会立刻知道你没漏东西。

- Let me map the brief's requirements to the design first, then go into each piece.
- One: every valid user and scrape date in the input appears in the output; bad scrapes dropped. I drop bad rows, seven rules, and drop a snapshot only if every row is bad. Coverage is 99.94 percent.
- Two: only current and past observations, never future ones; ids assigned on first sight and kept. That's the state machine fed in date order, plus the canary test.
- Three: no user–date pairs the input doesn't have. Output rows come only from an input snapshot.
- Four, Case A: a job the user deleted, or the scrape missed, stays in the output under the new date. Known positions are carried forward; and the data says this is the normal case, half the snapshots have one row.
- Five, Case B: the same job seen with different values across scrapes, RL versus Revelio Labs, MLE versus Machine Learning Engineer, a missing description. Rules 2 to 5 match the row to the existing id; absorb keeps the latest non–null value per field, so the description is the last one seen; and aliases remember every spelling for future matching.
- Six, other edge cases I found in the data: rehires, renamed employers, translated titles, garbled text, scrapes with every title null, duplicate ids. Each one became a rule and a regression test.
- Seven, scale and daily updates: 100 billion rows is about 800 core–hours with the same function on Spark; daily runs via saved state, late data

replays one person.

- On accuracy versus speed: the expensive part is fuzzy matching, so I only pay for it on the rows that need it. The fingerprint skips 57.5 percent of snapshots, rule 4 matches most of the rest without scoring, and only contested pairs get scored. Accuracy is measured on judged samples, speed on the full run.

Case A 例子：1 月快照有 Google 和 Meta 两段，3 月快照只抓到 Meta，3 月的输出里 Google 仍然在，id 不变。Case B 例子：1 月写 Machine Learning Engineer，3 月改成 MLE，同一个 id，头衔显示 MLE；描述 3 月是空的就保留 1 月的（取最新非空）。

他的评分三条：准确度（边界情况处理是否合理）、速度（1000 万行多快、1000 亿行要多少资源）、平衡说清楚。总结那页回到这三条。

Slide 2 · One state machine per person

- The core idea is one state machine per person. I sort a person's snapshots by date and feed them one at a time into a function called step.
- step only sees the current snapshot and the state built from earlier ones. So it cannot use future information, not because I'm careful, but by construction. There's also a test that plants a 2030 snapshot and checks it never shows up earlier.
- 57.5 percent of snapshots are identical to the previous one. I hash the rows into a fingerprint, and if it equals the last one I skip matching and just re-emit the positions with the new date.
- The fingerprint is a blake2b hash of the row contents, sorted first so that a reordered profile still hashes the same, and it's deterministic across processes so it can be saved in state. If two different snapshots ever collided, the only effect would be treating one as unchanged until the next change.

- Ids are user_id shifted left 16 bits plus a per-user sequence number. That's reproducible from the input, needs no global counter, and different users can never collide.

被问「指纹怎么算」：7 个字段 repr 成字符串、排序、blake2b 哈希成 64 位；排序是因为快照是集合，行序变了不算变；不用 Python 的 hash() 是因为它每个进程加随机盐，存进状态第二天读回来会对不上。

被问「为什么 65536」：数据里 user_id 最大 32 位，加 16 位共 48 位，64 位整数放得下；每人最多 166 个职位，99 分位 29，超了报错进隔离，绝不静默撞号。借鉴 Snowflake：把 64 位拆成几段、各管各的。

Slide 3 · Pipeline

- Reading from the top: conform renames company_website to company_linkedin_url, since the brief and the sample data disagree, fails fast if any of the ten columns is missing, casts types with bad values becoming null, and truncates text to 4,000 characters.
- Then I partition by user_id mod 64, one input file at a time, so a person's rows from different files end up in the same bucket.
- Each bucket is cleaned, seven row-level rules, and garbled text is repaired losslessly, about 16 thousand cells.
- Then the state machine runs per person, per snapshot in date order, and writes one output parquet per bucket.
- With state-out, each bucket also saves its state, which is what makes daily runs possible. One person's error is quarantined and that person rolled back; nobody else is affected.
- Concretely, `state-out` writes three parquet files per bucket: positions, every person's current positions with their alias sets; users, a small ledger per person with the next sequence number, the last fingerprint and the last date processed; and history, the cleaned input rows so far. Plus a meta file with the bucket count.

- state-in is the same folder read back the next day. The run first checks the bucket count in the meta file and refuses to start if it differs, because the same person would land in a different bucket and their state wouldn't be found.
- The CLI is just --input, --output, --state-in, --state-out, and --report-dir for metrics, quarantine and the restated list, so the output folder holds nothing but result files.

桶 = 一个文件夹，里面每个输入文件一个小 parquet；桶内按 user_id、observed_ts、order_in_profile 排序，桶和桶之间不排序（不需要，人和人独立）。桶数是参数，默认 64，不是写死：数据量和机器内存变了要能调；_meta.json 防止换了桶数还接着增量。

被问「为什么分桶」：内存有上限，一次只装一桶；人和人独立可并行；输入文件是按行切的不是按人切的，同一个人散在好几个文件里，所以必须先按 user_id 聚一次。

被问「清洗哪 7 条」：user_id 或日期为空；公司和头衔都空；结束早于开始；开始或结束早于 1950；开始晚于抓取 1 年以上；结束晚于抓取 2 年以上。README 写了 six，代码是 7 条，少写了一条。

Slide 4 · What the data said → what I did

- Before writing rules I looked at the data, and most decisions came from it.
- 52 percent of snapshots have a single row, and 30 percent of people never show more than one row. So "a job disappeared" usually means "this scrape only caught the top entry". Carrying positions forward, Case A, is the normal case, not the exception.
- 57.5 percent of snapshots repeat the previous one, hence the fingerprint.
- About 79 thousand groups share employer and start date but differ in title. Half are one job written twice, half are real parallel roles. So I can't decide on employer and start alone; I have to compare titles.
- About 20 percent of start dates are January 1st, which really means "only the year was entered". So two starts in the same year count as

compatible when either is January 1st.

- Some scrapes have every title null. A missing title matches any title, but only within that row, never stored as an alias. I'll come back to why.
- Garbled text, like UTF–8 read as Latin–1, and lost backslashes in escapes like Lloyd–u2019–s. I repair both losslessly during cleaning.
- Bad rows are under 0.1 percent, so I drop the row, not the whole snapshot. A snapshot is dropped only if every row is bad, 2,539 out of 3.9 million.

你的决定：只丢坏行不丢整次快照（「整次丢掉很可惜」）。

Slide 5 · Tech choices

- Polars for the data work. Not pandas: it's mostly single–threaded and strings are Python objects, so 2.5 gigabytes of input would blow up well past 16 gigabytes. Not DuckDB: the core is a per–person state machine with fuzzy matching, which is awkward in SQL and would mean two engines. Not local Spark: slow JVM start, a one–gigabyte–plus image, and slower than Polars on one machine.
- Polars hands Arrow tables to Spark with zero copy, so the same function runs in both. Spark is the production path.
- rapidfuzz, a C++ library, for title similarity. uv lock so the build is reproducible. Docker pinned by digest, non–root, input mounted read–only, and it runs with network disabled.
- One more: regular Polars needs AVX2 and hung under x86 emulation on my Mac, so the image uses the lts–cpu build.

Slide 6 · Six rules（总图）

- This is what happens to one snapshot of one person. First the fingerprint check. If it's new, rule 1 merges duplicate rows inside the snapshot. Rule 2 finds candidate positions for each row. Rule 3 excludes a rehire. Rule 4 matches an uncontested candidate directly. Rule 5 scores the contested ones. Then absorb updates fields and aliases, and rule 6 folds duplicate ids.
- Let me go through them one by one. The details are in the linked doc if you want to see each decision tree.

Rule 1 · Dedupe inside the snapshot

- People really do list one job twice in the same profile: two wordings like Sales Representative and Representatives, two titles for one role like Owner and Optometrist, a second-language copy of the profile, or a scrape duplicate. If I don't merge those first, the same job gets two ids.
- Two rows merge when three things hold: the same start date, or one side missing; the same employer, a shared URL or a normalised name that's equal or a whole-word prefix, like Honeywell and Honeywell Aerospace USA; and a matching title.
- Matching title means any of: identical or one side empty; similarity at least 90; the same description; or the same start and end date with a different title.
- Within one snapshot neither row is newer, so the first row's values win and later rows only fill empty fields. The group also remembers every title folded into it.
- Deliberately kept apart: same start, different end, different title. That's usually a promotion split into two entries.
- The same-start-and-end clause was added after my own labelling. I first said no to it, because I worried about real parallel roles. Then I labelled 60 pairs and every one of the 15 in that stratum was one job.

Adding it merged 2,963 more groups, and when I sampled the new merges, 55 of 71 were right. The misses are real simultaneous roles. Leaving them apart was wrong about three times as often as merging.

候选 = 「嫌疑名单」比方放在规则 2。被问头衔相似度：规范化 → 缩写 90 → 约 170 个职位词 9 种语言翻成英文、中日文按字切 → rapidfuzz token_set_ratio；不用 Jaccard 因为头衔太短，加个 Senior 就掉到 67，复数掉到 33。描述用 Jaccard ⩾ 0.5 因为是长文本。韩文词表在但编码范围漏了，实际没生效，一行修法。

Rule 2 · Candidates

- A candidate is a shortlist entry: an existing position that could plausibly be the same job as this row. It's not a match yet. Think of it as screening resumes before the interview; rules 4 and 5 are the interview.
- A position is a candidate if it shares the employer, under any name or URL it has ever had, and either the row's title is one the position has used, or the start is compatible: within 62 days, missing, or the same year with a January 1st.
- There's a fallback for renamed employers with no URL: identical title and identical start, different name. That came from a real case, Polar Truck becoming Polar Transportes Grupo DHL.
- Why shortlist first: scoring is the expensive part, and it also prevents matching across different employers just because titles look alike.

「同雇主看的是职位历史上用过的所有别名」这句要说，它是别名集合的用处。

Rule 3 · Rehire

- If a row starts after a position's known end date, it can't be an edit of that position. Someone who was a barista in 2019 to 2020 and came back in 2022 has two positions.

- Without this, the second stint would match the first and absorb would overwrite its dates, erasing the first one.
- It's not validation, the data is fine; it only decides which position a row belongs to. Both dates must be known; an open-ended position doesn't block.
- Effect: 1,579 more ids, 939 ids whose dates jumped around went to zero, 57 of 58 sampled were right, and it was 5 percent faster.

你的决定，中途改的主意：「我现在感觉更对」。

Rule 4 · One uncontested candidate

- If a row has exactly one candidate and no other row wants that candidate, I match it directly, without scoring, even if the title changed. That's how Case B keeps its id: Machine Learning Engineer becoming MLE at the same employer with nobody else competing.
- This is the main place the trade-off shows: when there's no competing explanation, merge.

Rule 5 · Scoring the contested ones

- When several rows or positions compete, I score every remaining pair: title similarity from 0 to 100, plus 25 for the same employer, 10 for the same start, 5 for the same end. Then I pair greedily from the highest score down.
- Below 55 the row opens a new position. At 120 or more, that's an identical title at the same employer, it joins a position even if another row already took it, because that's a scrape duplicate.
- Example: known Google, Software Engineer, March 2022. A row Senior Software Engineer, March 2022 scores 100 plus 25 plus 10, 135, match.

Product Manager, July 2023 isn't even a candidate: start is over a year away and the title was never used.

- Why 55: at the same employer that means a title similarity of at least 30; across employers it's almost impossible to reach. Why 120: 100 for an identical title plus 25 for the employer can only come from a true duplicate row.
- The trade-off throughout is merge when in doubt. A missed merge is a duplicate id carried forward forever; a wrong merge loses one title. The first was far more common.

被问「为什么 55」：同雇主时头衔至少 30 分才配；不同雇主几乎配不上。被问「为什么贪心」：一行只配一个职位、一个职位只给一行，分高的先定。

Rule 6 · Fold duplicate ids

- Rules 2 to 5 match one row to one position, so they can't merge two ids that were already issued. And aliases build up over time: early on, two ids can look like different jobs; a later snapshot adds a URL or a title that links them. Rule 6 is the repair step for that.
- After each snapshot, for positions touched this time, if another position has the same start, the same employer under any alias, and the same end or similar titles or the same description, the newer id folds into the older one and is retired for good.
- Real case: one profile wrote the same job as Jeppesen, Jeppesen Sanderson and Jeppesen slash Boeing and got five ids on the first scrape.
- Effect: 965 merges, suspected duplicate groups from 49 thousand down to 37 thousand, 52 of 64 sampled were right. The errors are two roles with identical dates, same trade-off as rule 1.
- One promise changed: an id never changes, but it can retire through a fold and is never reissued. Downstream has to handle an id that stops

appearing.

比方：前面是「新来的人分到哪个房间」，规则 6 是「发现两个房间住的是同一个人，合并房间」。

absorb · 字段怎么更新（讲完规则顺带）

- When a row matches a position, each field takes the latest non-null value, and the position remembers every employer name, URL and title it has had. Match on all aliases, display the latest.
- Two things I got wrong first. An empty end date was treated as "still ongoing" and it erased 43 thousand known end dates, because a degraded scrape simply drops fields. Now an empty end means "no information" and never overwrites a known end.
- And an empty title was being stored as an alias, so one degraded scrape made a position match every later title at that employer. A 2018 Consultant swallowed a 2024 Senior Director. Now a missing title is a wildcard for that row only.

结束日期这条是你一开始主张「空 = 至今」，数据证明错了才改的。

Slide 7 · Scale

- People are independent: nothing is shared between them, no global counter. So the work is linear in people and buckets run in parallel.
- Why buckets at all: memory is bounded by one bucket, and 64 buckets over 4 workers balances load, about 16 each.
- Why 64 and not 32: 32 ran out of memory in a 7.6 gigabyte container and only wrote 750 thousand rows. With 64, peak is about 2 gigabytes and it never crashed again.

- Why user_id mod 64 and not a library hash: state is saved per bucket, so the same person must land in the same bucket tomorrow. Polars' hash changes across versions; modulo doesn't. A meta file records the bucket count and the run refuses to start on a mismatch.
- Why a power of two: not required, but doubling splits each bucket cleanly in two, and it divides evenly across 1, 2, 4, 8 workers.
- Measured: 10 million rows in 75 to 78 seconds on 4 processes, about 2 gigabytes. In Docker under x86 emulation about 190 seconds, identical output.
- 100 billion rows: about 35 thousand rows per core–second, so roughly 800 core–hours, about 12 hours on 64 cores.
- Spark: the adapter is about 70 lines. It repartitions by user_id and calls the same coalesce_bucket through mapInArrow, so there's one copy of the logic. On two input files it matched the single–machine output row for row. I haven't run it on a real cluster, and incremental state isn't wired into it yet. At that scale I'd also raise the partition count so each partition stays at a few hundred thousand rows.
- In production the state belongs in a user–partitioned, append–only table like Iceberg or Delta: merge only the touched users, one atomic commit per run, old versions kept for rollback and as–of queries.

桶数不是越多越好：文件太小太多读写开销上来；Spark 版默认 4096。

Slide 7a · Three fixes

- Thread oversubscription: four workers each spinning up a full Polars thread pool, so the CPU spent most of its time switching. One Polars thread per worker took it from 104 to 65 seconds.
- The container was limited to 4 CPUs but os.cpu_count saw 14, so I got 14 workers and 5.5 gigabytes. Reading the cgroup CPU quota fixed that:

4 workers, 3.7 gigabytes.

- And the 32-to-64 bucket change I mentioned.
- Three smaller ones: regular Polars hung under x86 emulation, so the image uses the lts-cpu build; merging produced start-after-end dates, 733 of them, fixed by the end-date guard in absorb, now zero; and Spark executors picked up an old system Polars, fixed by pointing PYSPARK_PYTHON at the venv.

Slide 8 · Daily updates and late data

- state-out saves, per bucket, each person's positions with their aliases, a small ledger with the next sequence number, last fingerprint and last date, plus the cleaned history. state-in loads it the next day.
- Only people with new snapshots are processed, continuing from where they left off, same ids.
- Late data means a snapshot dated on or before what I've already processed for that person: a crawler retry, a weekly vendor drop, a backfill. Feeding it into step would break date order and could overwrite newer values with older ones. So I replay just that person from history, re-emit all their rows, and list them under restated so downstream replaces them. Without history they're quarantined.
- Tests: a full run equals a half run plus an incremental run on the other half, which also proves no future data is used. And 128 late users replayed on real data matched a full run exactly.
- I started with quarantining late data and changed to replay: late data is routine, quarantine leaves that person's output wrong until someone looks, and replaying one person is cheap and verifiable.
- Why not just quarantine late data: it's routine, quarantine leaves that person's output wrong until someone looks, replaying one person is

cheap, and the result can be checked against a full run.

- Known weak spot: today I load a bucket's whole history every run and rewrite it. That grows daily. The fix is to append one file per day and read history only for the few late users.

Slide 9 · Evaluation

- There's no ground truth, so I built three kinds of evidence.
- First, LLM judges. I sampled 75 decisions from each of four strata, showed both records and asked "one entry or two". Same prompt to Claude and GPT, all 300 pairs each, repeated after every rule change. The merge strata are at 95 to 97 percent. New id at a known employer, 97 and 87. The weak stratum is "same employer and start, kept apart": 38 and 8 percent. It's 0.26 percent of ids.
- Second, 60 pairs I labelled by hand. 52 of 60 agree with Claude, kappa 0.66, and every disagreement is in that weak stratum, which I read as one entry every time.
- Third, label–free checks counted on the output: coverage 99.94 percent, a unique primary key, zero inverted dates, and suspicious splits, one person, date, employer and start with two or more ids, down from 267 thousand with exact matching to 37 thousand, each step tied to one rule change.
- What each caught: a judge pass flagged three pairs with start after end, which led me to the two absorb bugs. My own labelling caught the garbled text the judges had passed, and it changed the same–start–and–end rule.
- What the judges are for, concretely: measure precision per stratum; surface the misjudged examples so I can read the raw data, change one rule and add one regression test; A/B every rule change by judging the

newly merged and newly split pairs; and cross-check each other with the same prompt, kappa 0.66, calibrated against my 60 hand labels.

- The judges are offline graders. They're not in the pipeline and I didn't train anything on them, except one analysis: a logistic regression on 823 labelled pairs, which learned the same ordering as my rules, start date, then end, then description, then title, and only added 6 points of recall at equal precision on a biased sample. So I kept it as evidence rather than shipping it.
- 51 tests overall, including the canary and about 20 regressions from real profiles.

被问 26.7 万 → 3.7 万怎么来的：第一版只精确匹配 26.7 万 → 四条漏配修法（改名兜底、URL 规范化、去重用前缀判雇主、去重不看结束）15.5 万 → 空头衔本行通配 12.6 万 → bug 修复 + Unicode + 乱码约 10.5 万 → 同开始同结束合并 5.5 万 → 微调 4.9 万 → 规则 6 折叠 3.7 万。

两个 bug 的发现路径：第二轮 Claude 判官标 3 对「开始晚于结束」（Mercado Libre、Alcon、Veeva）→ 输出里明明没有倒序 → 顺着 Veeva 查输入 → 三次降级抓取只剩一行、头衔开始结束全空 → 并进 Consultant → 两个 bug 同时触发。修完抹掉的结束日期 43,356 → 53，两个回归测试。

人工评估解决了什么：发现乱码（16,314 格）、发现评估数据缺字段、推翻一条规则（同开始同结束合并）、校准判官（52/60，kappa 0.66，分歧全在弱点层）。没解决的：弱点层判官之间也不一致（Claude 38%、GPT 8%），要更多标注。

被问 kappa：判官之间的一致性，去掉碰巧一致的部分；0.66 算 substantial。

Slide 10 · What I tried

- Identity by latest value only: Febase renamed their employer and later renamed it back, and became two ids. That's why positions keep alias sets.
- ASCII-only normalisation: 228 thousand non-Latin titles became empty strings, and empty matched anything. Fixed with Unicode normalisation.
- Multilingual embeddings: 383 more groups merged, 72 of 75 right, but three times the run time and 1.2 gigabytes more image. Off by default; a

glossary of about 170 job words gets the same gain for translations with no dependency.

- Descriptions as a signal: identical text is the same job 99 percent of the time, but different text is still the same job 82 percent of the time. So positive evidence only.
- The learned scorer I just mentioned. And rehire as a new position, and folding duplicate ids, both kept.

Slide 11 · Summary

- One state machine per person, fed in date order, so no future information by construction.
- Six rules and a score; merge when in doubt.
- People are independent, so bucket by user_id, linear scale, the same function on Spark.
- Daily runs touch only people with new data; late data replays one person.
- No ground truth, so judges plus hand labels plus label–free checks, and every finding became a rule and a test.
- Why not an LLM on every pair: at 100 billion rows that's not practical, ids must be reproducible and an LLM's answers aren't, and rules are explainable, each error points to one rule. So the LLM sits offline as a grader on samples, not on the hot path.

Slide 12 · Next steps

- Distil the judge and human labels into a small scorer, used only where the rules are unsure, the kept–apart stratum. More labels there.

- Precompute description embeddings offline.
- Spark on a real cluster with incremental state, in an append-only user-partitioned table.
- A deletion list for right-to-be-forgotten requests, because carrying positions forward keeps entries a user deleted.
- Small ones: the Korean glossary range, and initials matching for employer names, Revelio Labs versus RL.

被问「怎么用 AI」

- AI wrote code from my designs, drafted tests, ran exploration scripts, labelled pairs as a judge, and traced failures I pointed at.
- I decided every rule and threshold, defined what "same job" means, designed the sampling, and labelled 60 pairs by hand.
- The decisions I changed along the way: drop rows not snapshots, the same-start-and-end merge after labelling, rehire as a new id, replay instead of quarantine for late data.
- The garbled text came from my own labelling, not the judge, and my labels changed one of the rules.